Beej's Guide to Network Programming

Using Internet Sockets

Brian "Beej Jorgensen" Hall

v3.3.2, Copyright © April 18, 2026

Contents

1 Intro **1**
- 1.1 Audience . . . 1
- 1.2 Platform and Compiler . . . 1
- 1.3 Official Homepage and Books For Sale . . . 1
- 1.4 Note for Solaris/SunOS/illumos Programmers . . . 2
- 1.5 Note for Windows Programmers . . . 2
- 1.6 Email Policy . . . 4
- 1.7 Mirroring . . . 4
- 1.8 Note for Translators . . . 4
- 1.9 Copyright, Distribution, and Legal . . . 5
- 1.10 Dedication . . . 5
- 1.11 Publishing Information . . . 5

2 What is a socket? **6**
- 2.1 Two Types of Internet Sockets . . . 6
- 2.2 Low level Nonsense and Network Theory . . . 8

3 IP Addresses, `struct`s, and Data Munging **10**
- 3.1 IP Addresses, versions 4 and 6 . . . 10
  - 3.1.1 Subnets . . . 11
  - 3.1.2 Port Numbers . . . 12
- 3.2 Byte Order . . . 12
- 3.3 `struct`s . . . 14
- 3.4 IP Addresses, Part Deux . . . 16
  - 3.4.1 Private (Or Disconnected) Networks . . . 17

4 Jumping from IPv4 to IPv6 **19**

5 System Calls or Bust **21**
- 5.1 `getaddrinfo()`—Prepare to launch! . . . 21
- 5.2 `socket()`—Get the File Descriptor! . . . 25
- 5.3 `bind()`—What port am I on? . . . 26
- 5.4 `connect()`—Hey, you! . . . 27
- 5.5 `listen()`—Will somebody please call me? . . . 28
- 5.6 `accept()`—"Thank you for calling port 3490." . . . 29
- 5.7 `send()` and `recv()`—Talk to me, baby! . . . 30
- 5.8 `sendto()` and `recvfrom()`—Talk to me, DGRAM-style . . . 32
- 5.9 `close()` and `shutdown()`—Get outta my face! . . . 32
- 5.10 `getpeername()`—Who are you? . . . 33
- 5.11 `gethostname()`—Who am I? . . . 34

6 Client-Server Background **35**

6.1 A Simple Stream Server . . . 35
6.2 A Simple Stream Client . . . 38
6.3 Datagram Sockets . . . 41

7 Slightly Advanced Techniques 45
7.1 Blocking . . . 45
7.2 `poll()`—Synchronous I/O Multiplexing . . . 46
7.3 `select()`—Synchronous I/O Multiplexing, Old School . . . 53
7.4 Handling Partial `send()`s . . . 61
7.5 Serialization—How to Pack Data . . . 62
7.6 Son of Data Encapsulation . . . 76
7.7 Broadcast Packets—Hello, World! . . . 77

8 Common Questions 81

9 Man Pages 87
9.1 `accept()` . . . 88
9.2 `bind()` . . . 89
9.3 `connect()` . . . 91
9.4 `close()` . . . 92
9.5 `getaddrinfo()`, `freeaddrinfo()`, `gai_strerror()` . . . 93
9.6 `gethostname()` . . . 96
9.7 `gethostbyname()`, `gethostbyaddr()` . . . 97
9.8 `getnameinfo()` . . . 99
9.9 `getpeername()` . . . 101
9.10 `errno` . . . 102
9.11 `fcntl()` . . . 103
9.12 `htons()`, `htonl()`, `ntohs()`, `ntohl()` . . . 104
9.13 `inet_ntoa()`, `inet_aton()`, `inet_addr` . . . 105
9.14 `inet_ntop()`, `inet_pton()` . . . 106
9.15 `listen()` . . . 109
9.16 `perror()`, `strerror()` . . . 110
9.17 `poll()` . . . 111
9.18 `recv()`, `recvfrom()` . . . 113
9.19 `select()` . . . 115
9.20 `setsockopt()`, `getsockopt()` . . . 117
9.21 `send()`, `sendto()` . . . 119
9.22 `shutdown()` . . . 121
9.23 `socket()` . . . 121
9.24 `struct sockaddr` and pals . . . 123

10 More References 126
10.1 Books . . . 126
10.2 Web References . . . 126
10.3 RFCs . . . 127

Chapter 1

Intro

Hey! Socket programming got you down? Is this stuff just a little too difficult to figure out from the `man` pages? You want to do cool Internet programming, but you don't have time to wade through a gob of `struct`s trying to figure out if you have to call `bind()` before you `connect()`, etc., etc.

Well, guess what! I've already done this nasty business, and I'm dying to share the information with everyone! You've come to the right place. This document should give the average competent C programmer the edge s/he needs to get a grip on this networking noise.

And check it out: I've finally caught up with the future (just in the nick of time, too!) and have updated the Guide for IPv6! Enjoy!

1.1 Audience

This document has been written as a tutorial, not a complete reference. It is probably at its best when read by individuals who are just starting out with socket programming and are looking for a foothold. It is certainly not the *complete and total* guide to sockets programming, by any means.

Hopefully, though, it'll be just enough for those man pages to start making sense... `:-)`

1.2 Platform and Compiler

The code contained within this document was compiled on a Linux PC using Gnu's `gcc` compiler. It should, however, build on just about any platform that uses `gcc`. Naturally, this doesn't apply if you're programming for Windows—see the section on Windows programming, below.

1.3 Official Homepage and Books For Sale

This official location of this document is:

- `https://beej.us/guide/bgnet/`

There you will also find example code and translations of the guide into various languages.

To buy nicely bound print copies (some call them "books"), visit:

- `https://beej.us/guide/url/bgbuy`

I'll appreciate the purchase because it helps sustain my document-writing lifestyle!

1.4 Note for Solaris/SunOS/illumos Programmers

When compiling for a Solaris variant or SunOS, you need to specify some extra command-line switches for linking in the proper libraries. In order to do this, simply add "`-lnsl -lsocket -lresolv`" to the end of the compile command, like so:

```
$ cc -o server server.c -lnsl -lsocket -lresolv
```

If you still get errors, you could try further adding a `-lxnet` to the end of that command line. I don't know what that does, exactly, but some people seem to need it.

Another place that you might find problems is in the call to `setsockopt()`. The prototype differs from that on my Linux box, so instead of:

```
int yes=1;
```

enter this:

```
char yes='1';
```

As I don't have a Sun box, I haven't tested any of the above information—it's just what people have told me through email.

1.5 Note for Windows Programmers

At this point in the guide, historically, I've done a bit of bagging on Windows, simply due to the fact that I don't like it very much. But then Windows and Microsoft (as a company) got a lot better. Windows 10 coupled with WSL (below) actually makes for a decent operating system. Not really a lot to complain about.

Well, a little—for example, I'm writing this (in 2025) on a 2015 laptop that used to run Windows 10. Eventually it got too slow and I installed Linux on it. And have been using it ever since.

But now we have Windows 11 that apparently requires beefier hardware than Windows 10. I'm not a fan of that. The OS should be as unobtrusive as possible and not require you to spend more money. The extra CPU power should be for apps, not the OS! Additionally, Microsoft knows what you want, and what you want is more advertising! Right? In your operating system! Weren't you missing that? Now you can have it with Windows 11.

So... I still encourage you to try Linux[1], BSD[2], illumos[3] or any other flavor of Unix instead of Windows.

How'd that soapbox get there?

But people like what they like, and you Windows folk will be pleased to know that this information is generally applicable to Windows, with a few minor changes.

One thing that you should strongly consider is the Windows Subsystem for Linux[4]. This basically allows you to install a Linux VM-ish thing on Windows 10. That will also definitely get you situated, and you'll be able to build and run these programs as is.

Another thing you can do is install Cygwin[5], which is a collection of Unix tools for Windows. I've heard on the grapevine that doing so allows all these programs to compile unmodified, but I've never tried it.

[1] https://www.linux.com/
[2] https://bsd.org/
[3] https://www.illumos.org/
[4] https://learn.microsoft.com/en-us/windows/wsl/
[5] https://cygwin.com/

Some of you might want to do things the Pure Windows Way. That's very gutsy of you, and this is what you have to do: run out and get Unix immediately! No, no—I'm kidding. I'm supposed to be Windows-friendly(er) these days...

Okay, okay. I'll get on with it.

This is what you'll have to do: first, ignore pretty much all of the system header files I mention in here. Instead, include:

```
#include <winsock2.h>
#include <ws2tcpip.h>
```

`winsock2` is the "new" (circa 1994) version of the Windows socket library.

Unfortunately, if you include `windows.h`, it automatically pulls in the older `winsock.h` (version 1) header file which conflicts with `winsock2.h`! Fun times.

So if you have to include `windows.h`, you need to define a macro to get it to *not* include the older header:

```
#define WIN32_LEAN_AND_MEAN  // Say this...

#include <windows.h>         // And now we can include that.
#include <winsock2.h>        // And this.
```

Wait! You also have to make a call to `WSAStartup()` before doing anything else with the sockets library. You pass in the Winsock version you desire to this function (e.g. version 2.2). And then you can check the result to make sure that version is available.

The code to do that looks something like this:

```
#include <winsock2.h>

{
    WSADATA wsaData;

    if (WSAStartup(MAKEWORD(2, 2), &wsaData) != 0) {
        fprintf(stderr, "WSAStartup failed.\n");
        exit(1);
    }

    if (LOBYTE(wsaData.wVersion) != 2 ||
        HIBYTE(wsaData.wVersion) != 2)
    {
        fprintf(stderr,"Version 2.2 of Winsock not available.\n");
        WSACleanup();
        exit(2);
    }
```

Note that call to `WSACleanup()` in there. That's what you want to call when you're done with the Winsock library.

You also have to tell your compiler to link in the Winsock library, called `ws2_32.lib` for Winsock 2. Under VC++, this can be done through the `Project` menu, under `Settings...`. Click the `Link` tab, and look for the box titled "Object/library modules". Add "ws2_32.lib" (or whichever lib is your preference) to that list.

Or so I hear.

Once you do that, the rest of the examples in this tutorial should generally apply, with a few exceptions. For one thing, you can't use `close()` to close a socket—you need to use `closesocket()`, instead. Also, `select()` only works with socket descriptors, not file descriptors (like `0` for `stdin`).

There is also a socket class that you can use, `CSocket` Check your compiler's help pages for more information.

To get more information about Winsock, check out the official page at Microsoft.

Finally, I hear that Windows has no `fork()` system call which is, unfortunately, used in some of my examples. Maybe you have to link in a POSIX library or something to get it to work, or you can use `CreateProcess()` instead. `fork()` takes no arguments, and `CreateProcess()` takes about 48 billion arguments. If you're not up to that, the `CreateThread()` is a little easier to digest…unfortunately a discussion about multithreading is beyond the scope of this document. I can only talk about so much, you know!

Extra finally, Steven Mitchell has ported a number of the examples[6] to Winsock. Check that stuff out.

1.6 Email Policy

I'm generally available to help out with email questions so feel free to write in, but I can't guarantee a response. I lead a pretty busy life and there are times when I just can't answer a question you have. When that's the case, I usually just delete the message. It's nothing personal; I just won't ever have the time to give the detailed answer you require.

As a rule, the more complex the question, the less likely I am to respond. If you can narrow down your question before mailing it and be sure to include any pertinent information (like platform, compiler, error messages you're getting, and anything else you think might help me troubleshoot), you're much more likely to get a response. For more pointers, read ESR's document, How To Ask Questions The Smart Way[7].

If you don't get a response, hack on it some more, try to find the answer, and if it's still elusive, then write me again with the information you've found and hopefully it will be enough for me to help out.

Now that I've badgered you about how to write and not write me, I'd just like to let you know that I *fully* appreciate all the praise the guide has received over the years. It's a real morale boost, and it gladdens me to hear that it is being used for good! `:-)` Thank you!

1.7 Mirroring

You are more than welcome to mirror this site, whether publicly or privately. If you publicly mirror the site and want me to link to it from the main page, drop me a line at `beej@beej.us`.

1.8 Note for Translators

If you want to translate the guide into another language, write me at `beej@beej.us` and I'll link to your translation from the main page. Feel free to add your name and contact info to the translation.

This source markdown document uses UTF-8 encoding.

Please note the license restrictions in the Copyright, Distribution, and Legal section, below.

If you want me to host the translation, just ask. I'll also link to it if you want to host it; either way is fine.

[6] https://www.tallyhawk.net/WinsockExamples/
[7] http://www.catb.org/~esr/faqs/smart-questions.html

1.9 Copyright, Distribution, and Legal

Beej's Guide to Network Programming is Copyright © 2019 Brian "Beej Jorgensen" Hall.

With specific exceptions for source code and translations, below, this work is licensed under the Creative Commons Attribution- Noncommercial- No Derivative Works 3.0 License. To view a copy of this license, visit

`https://creativecommons.org/licenses/by-nc-nd/3.0/`

or send a letter to Creative Commons, 171 Second Street, Suite 300, San Francisco, California, 94105, USA.

One specific exception to the "No Derivative Works" portion of the license is as follows: this guide may be freely translated into any language, provided the translation is accurate, and the guide is reprinted in its entirety. The same license restrictions apply to the translation as to the original guide. The translation may also include the name and contact information for the translator.

The C source code presented in this document is hereby granted to the public domain, and is completely free of any license restriction.

Educators are freely encouraged to recommend or supply copies of this guide to their students.

Unless otherwise mutually agreed by the parties in writing, the author offers the work as-is and makes no representations or warranties of any kind concerning the work, express, implied, statutory or otherwise, including, without limitation, warranties of title, merchantability, fitness for a particular purpose, noninfringement, or the absence of latent or other defects, accuracy, or the presence of absence of errors, whether or not discoverable.

Except to the extent required by applicable law, in no event will the author be liable to you on any legal theory for any special, incidental, consequential, punitive or exemplary damages arising out of the use of the work, even if the author has been advised of the possibility of such damages.

Contact `beej@beej.us` for more information.

1.10 Dedication

Thanks to everyone who has helped in the past and future with me getting this guide written. And thank you to all the people who produce the Free software and packages that I use to make the Guide: GNU, Linux, Slackware, vim, Python, Inkscape, pandoc, many others. And finally a big thank-you to the literally thousands of you who have written in with suggestions for improvements and words of encouragement.

I dedicate this guide to some of my biggest heroes and inpirators in the world of computers: Donald Knuth, Bruce Schneier, W. Richard Stevens, and The Woz, my Readership, and the entire Free and Open Source Software Community.

1.11 Publishing Information

This book is written in Markdown using the vim editor on an Arch Linux box loaded with GNU tools. The cover "art" and diagrams are produced with Inkscape. The Markdown is converted to HTML and LaTex/PDF by Python, Pandoc and XeLaTeX, using Liberation fonts. The toolchain is composed of 100% Free and Open Source Software.

Chapter 2

What is a socket?

You hear talk of "sockets" all the time, and perhaps you are wondering just what they are exactly. Well, they're this: a way to speak to other programs using standard Unix file descriptors.

What?

Ok—you may have heard some Unix hacker state, "Jeez, *everything* in Unix is a file!" What that person may have been talking about is the fact that when Unix programs do any sort of I/O, they do it by reading or writing to a file descriptor. A file descriptor is simply an integer associated with an open file. But (and here's the catch), that file can be a network connection, a FIFO, a pipe, a terminal, a real on-the-disk file, or just about anything else. Everything in Unix *is* a file! So when you want to communicate with another program over the Internet you're gonna do it through a file descriptor, you'd better believe it.

"Where do I get this file descriptor for network communication, Mr. Smarty-Pants?" is probably the last question on your mind right now, but I'm going to answer it anyway: You make a call to the `socket()` system routine. It returns the socket descriptor, and you communicate through it using the specialized `send()` and `recv()` (`man send`, `man recv`) socket calls.

"But, hey!" you might be exclaiming right about now. "If it's a file descriptor, why in the name of Neptune can't I just use the normal `read()` and `write()` calls to communicate through the socket?" The short answer is, "You can!" The longer answer is, "You can, but `send()` and `recv()` offer much greater control over your data transmission."

What next? How about this: there are all kinds of sockets. There are DARPA Internet addresses (Internet Sockets), path names on a local node (Unix Sockets), CCITT X.25 addresses (X.25 Sockets that you can safely ignore), and probably many others depending on which Unix flavor you run. This document deals only with the first: Internet Sockets.

2.1 Two Types of Internet Sockets

What's this? There are two types of Internet sockets? Yes. Well, no. I'm lying. There are more, but I didn't want to scare you. I'm only going to talk about two types here. Except for this sentence, where I'm going to tell you that "Raw Sockets" are also very powerful and you should look them up.

All right, already. What are the two types? One is "Stream Sockets"; the other is "Datagram Sockets", which may hereafter be referred to as "`SOCK_STREAM`" and "`SOCK_DGRAM`", respectively. Datagram sockets are sometimes called "connectionless sockets". (Though they can be `connect()`'d if you really want. See `connect()`, below.)

Stream sockets are reliable two-way connected communication streams. If you output two items into the socket in the order "1, 2", they will arrive in the order "1, 2" at the opposite end. They will also be error-free.

I'm so certain, in fact, they will be error-free, that I'm just going to put my fingers in my ears and chant *la la la la* if anyone tries to claim otherwise.

What uses stream sockets? Well, you may have heard of the `telnet` or `ssh` applications, yes? They use stream sockets. All the characters you type need to arrive in the same order you type them, right? Also, web browsers use the Hypertext Transfer Protocol (HTTP) which uses stream sockets to get pages. Indeed, if you telnet to a web site on port 80, and type "`GET / HTTP/1.0`" and hit RETURN twice, it'll dump the HTML back at you!

> If you don't have `telnet` installed and don't want to install it, or your `telnet` is being picky about connecting to clients, the guide comes with a `telnet`-like program called `telnot`[a]. This should work well for all the needs of the guide. (Note that telnet is actually a spec'd networking protocol[b], and `telnot` doesn't implement this protocol at all.)
>
> [a] https://beej.us/guide/bgnet/source/examples/telnot.c
> [b] https://tools.ietf.org/html/rfc854

How do stream sockets achieve this high level of data transmission quality? They use a protocol called "The Transmission Control Protocol", otherwise known as "TCP" (see RFC 793[1] for extremely detailed info on TCP). TCP makes sure your data arrives sequentially and error-free. You may have heard "TCP" before as the better half of "TCP/IP" where "IP" stands for "Internet Protocol" (see RFC 791[2]). IP deals primarily with Internet routing and is not generally responsible for data integrity.

Cool. What about Datagram sockets? Why are they called connectionless? What is the deal, here, anyway? Why are they unreliable? Well, here are some facts: if you send a datagram, it may arrive. It may arrive out of order. If it arrives, the data within the packet will be error-free.

Datagram sockets also use IP for routing, but they don't use TCP; they use the "User Datagram Protocol", or "UDP" (see RFC 768[3]).

Why are they connectionless? Well, basically, it's because you don't have to maintain an open connection as you do with stream sockets. You just build a packet, slap an IP header on it with destination information, and send it out. No connection needed. They are generally used either when a TCP stack is unavailable or when a few dropped packets here and there don't mean the end of the Universe. Sample applications: `tftp` (trivial file transfer protocol, a little brother to FTP), `dhcpcd` (a DHCP client), multiplayer games, streaming audio, video conferencing, etc.

"Wait a minute! `tftp` and `dhcpcd` are used to transfer binary applications from one host to another! Data can't be lost if you expect the application to work when it arrives! What kind of dark magic is this?"

Well, my human friend, `tftp` and similar programs have their own protocol on top of UDP. For example, the tftp protocol says that for each packet that gets sent, the recipient has to send back a packet that says, "I got it!" (an "ACK" packet). If the sender of the original packet gets no reply in, say, five seconds, he'll re-transmit the packet until he finally gets an ACK. This acknowledgment procedure is very important when implementing reliable `SOCK_DGRAM` applications.

For unreliable applications like games, audio, or video, you just ignore the dropped packets, or perhaps try to cleverly compensate for them. (Quake players will know the manifestation of this effect by the technical term: *accursed lag*. The word "accursed", in this case, represents any extremely profane utterance.)

Why would you use an unreliable underlying protocol? Two reasons: speed and speed. It's way faster to fire-and-forget than it is to keep track of what has arrived safely and make sure it's in order and all that. If you're sending chat messages, TCP is great; if you're sending 40 positional updates per second of the players in the world, maybe it doesn't matter so much if one or two get dropped, and UDP is a good choice.

[1] https://tools.ietf.org/html/rfc793
[2] https://tools.ietf.org/html/rfc791
[3] https://tools.ietf.org/html/rfc768